PROJECT ATHENA

v2.7

MASTER ARCHITECTURE & ENGINEERING SPECIFICATION

Risk-First Autonomous Trading OS | Persistent Market Experience | Governed Intelligence | Multi-Asset Execution

Architecture position

ATHENA v2.7 is the implementation baseline for the project. It supersedes the v2.6 conceptual baseline and incorporates the hardened AMEF contracts, Intelligence/Execution plane separation, deterministic Phase 1 context, version lineage, and the new TypeScript/React/Node.js monolithic frontend architecture. The frontend is an operational cockpit; Python remains the authoritative trading and risk backend.

Metadata	Specification
Document type	Master Architecture & Engineering Specification
Version	2.7
Status	Implementation Baseline
Supersedes	ATHENA v2.6 Master Architecture & Engineering Specification
Primary deployment target	Oracle VPS with GitHub-based CI/CD
Primary broker integration	MetaTrader 5 via Python adapter
Initial asset	XAUUSD
Planned expansion	12 major Forex pairs and selected indices
Execution modes	Scalping + Day Trading; alternative execution remains isolated/gated
Frontend	TypeScript + React, Node.js ecosystem, single frontend node_modules
Backend	Python
Persistence	PostgreSQL canonical store; pgvector as rebuildable AMEF retrieval layer
Governance	Risk-first, NO_TRADE-first, auditable, versioned, replayable

Design maxim: Build the foundation first. Measure everything. Remember context. Adapt carefully. Let risk remain the final authority.

0. Document Map and Use

This document is the working architectural contract for ATHENA v2.7. It is intentionally comprehensive because the goal is to stop architectural drift during implementation. Future revisions should modify explicit contracts and decisions rather than replacing the system design informally.

Section	Title	Purpose
1	Executive Summary & Design Intent	Purpose, scope, thesis, objectives, boundaries
2	Changes from v2.6	New architecture and hardening decisions
3	Principles, Constraints & Non-Goals	Non-negotiable behavior
4	System Context & High-Level Architecture	Five-plane architecture and runtime boundaries
5	Frontend Trading OS Architecture	React/TypeScript/Node.js, navigation, authentication
6	Python Backend Architecture	Core services, event bus, state, strategy boundaries
7	Account Management & Settings Architecture	Global/account/strategy/asset/session policy hierarchy
8	Market Context & Regime Engine	HTF context, regimes, news, volatility
9	Scalping & Day Trading Engines	Concurrent strategy operation with separated risk state
10	ATHENA Memory & Experience Fabric (AMEF)	Memory domains, evidence authority, retrieval
11	POI / Order Block Lifecycle	Formation, monitoring, reaction, decay, invalidation
12	Decision, Attribution & Counterfactual Learning	Why trade, why not, outcome attribution
13	Machine Learning & Calibration	Shadow-first XGBoost and model governance
14	Risk Governance & Account Guardian	Global and account-specific circuit breakers
15	Correlation & Portfolio Exposure	Multi-asset risk constraints
16	Execution & Broker Interfaces	ExecutionIntent, broker truth, reconciliation
17	Data, Persistence & Schema Lineage	PostgreSQL, pgvector, event semantics
18	Event-Driven 15-Stage Decision Pipeline	Canonical runtime flow
19	Observability, Executioner & Decision Stream	Instrument tabs, structured decision trace
20	Journal & Continuous Trading Record	Living journal and lifecycle states
21	Backtesting & Research Platform	Replay, walk-forward, memory ablation
22	CI/CD, Git, Deployment & Operations	State-safe delivery
23	Testing & Validation	Unit, contract, integration, chaos, replay
24	Failure Modes & Degraded Operation	Fail-open/fail-closed rules
25	Security & Access Control	Auth, secrets, sessions, audit integrity
26	Performance & Governance Metrics	Runtime and model governance metrics
27	Phased Development Roadmap	Revised implementation sequence
28	Definition of Done & Release Gates	Phase exit criteria
Appendix A	Core Data Contracts	ExecutionIntent, EpisodeRecord, EvidenceSnapshot
Appendix B	Enumerations & Reason Codes	Canonical machine-readable statuses
Appendix C	Conceptual Repository Structure	Monolithic repository layout
Appendix D	Key Equations & Design Constraints	Reference formulas and hard constraints
Appendix E	Architectural Decisions Register	v2.7 ADRs
Appendix F	Review-Driven Change Log	How v2.7 resolves prior review findings

1. Executive Summary & Design Intent

Project ATHENA is an event-driven algorithmic trading operating system designed to evolve the legacy CyBot lineage from a command-line trading bot into a modular, observable, risk-governed trading platform. The core backend is Python. The user-facing Trading OS is a TypeScript/React application operating within the Node.js ecosystem. The two sides communicate through explicit API and real-time event contracts.

ATHENA combines multi-timeframe Smart Money Concepts (SMC), deterministic market context, probabilistic machine learning, macro/news context, execution-cost awareness, portfolio risk controls, persistent market experience, and controlled adaptive learning. The system is deliberately designed so that no single intelligent subsystem becomes the final authority over risk.

Core thesis

ATHENA is a risk-first decision system, not a signal generator. It should be able to explain what it observed, what state it believed the market was in, what candidate setups were available, why it selected or rejected a candidate, what the account risk policy allowed, what the broker actually did, and what subsequently happened.

1.1 Primary objectives

- Preserve capital through deterministic hard safety gates and an explicit NO_TRADE outcome.
- Separate market context/regime classification from micro entry execution.
- Operate scalping and day trading concurrently when policy allows rather than treating them as mutually exclusive modes.
- Treat SMC concepts such as BOS, CHoCH, OB and FVG as versioned structural features and stateful POIs rather than binary labels.
- Introduce ML as a probabilistic filter only after the deterministic foundation proves replayability and data integrity.
- Create persistent contextual memory through AMEF without turning historical win rates into autonomous commands.
- Maintain a living journal that begins when an asset is scanned and evolves as fields become observable.
- Provide an Executioner interface that exposes structured, auditable decision events instead of an uncontrolled raw chain-of-thought stream.
- Support account-specific policy, including funded-account protection limits that can be stricter than the external firm requirements.
- Maintain complete execution and attribution records so that a strategy error, broker error and market outcome are not confused.
- Allow future multi-asset and multi-account operation without redesigning the core architecture.
- Provide a governed path from deterministic baseline to adaptive intelligence and champion/challenger production models.

1.2 The four questions ATHENA must continuously answer

Question	System responsibility	Primary surface
What is happening?	Data + market context + regime + SMC	Dashboard / Executioner / Journal
What does ATHENA conclude?	Candidate generation + ML + AMEF + decision logic	Executioner / audit record
What is ATHENA allowed to do?	Account Guardian + portfolio risk + execution policy	Settings / Dashboard / Executioner
What happened and what was learned?	Broker truth + outcome attribution + journal + AMEF	Journal / Governance / Research

1.3 Primary outputs

Output	Meaning	Priority
BUY / SELL	Executable directional decision after all required gates	Conditional
MODIFY	Approved change to an existing position	Conditional
NO_TRADE	Explicit rejection with reason codes	Primary valid outcome
OBSERVE	Candidate monitored but not executable	Valid
HALT	System/module/account safety lock	Highest
BLOCKED	Opportunity exists but policy/risk/account rule prevents execution	Valid and auditable
MEMORY_EVIDENCE	Historical contextual evidence attached to a candidate	Supporting

2. What Changed from v2.6

v2.7 preserves the strongest elements of v2.6 but turns several conceptual requirements into explicit implementation contracts. It also incorporates the newly clarified frontend and operational design. The changes below are normative unless a future version explicitly supersedes them.

Area	v2.6	v2.7 hardened position
Frontend	Kivy/native + Django web concept	TypeScript/React Trading OS with Node.js ecosystem; Python authoritative backend
Repository	Backend/frontend modules were conceptually separated	Modular monorepo: root backend/, frontend/, .venv/; one frontend node_modules
Authentication	Local auth listed as a feature	Dedicated frontend authentication module: Splash, Login, Sign Up, Forgot Password, with backend authorization/session authority
Settings	Baseline configuration surface	Hierarchical global -> account -> strategy -> asset -> session policy with account guardian
Journal	Audit trail mainly after execution	Living journal starts at scan and fills fields as evidence becomes available
Executioner	Telemetry-focused	Instrument tab workspace + structured Analysis/Decision Stream + why trade/why not + timeline
Reasoning UI	Potentially implied raw reasoning	Structured decision trace only; no requirement to expose hidden chain-of-thought
AMEF retrieval	Conceptual retrieval and weighting	Explicit filters, effective sample size, authority states, EvidenceSnapshot, rebuildable index
Runtime	15 stages could be interpreted as synchronous	Intelligence/Execution plane split; execution consumes valid versioned EvidenceSnapshot
Regime	Advanced regime later	Deterministic baseline regime context begins in Phase 1
Cold start	Minimum rules	Memory authority = 0 until sufficiency/reliability gates are passed
Counterfactuals	Bounded concept	Top-K rejected candidates, explicit cost model, permanently hypothetical flag
Persistence	DB + vector layer	PostgreSQL is canonical; pgvector is rebuildable acceleration
Governance	Model lifecycle	Version hashes, champion/challenger, policy lineage, schema compatibility

		gates
Roadmap	Six broad phases	Seven phases including a deliberate advanced adaptive intelligence stage

3. System Principles, Constraints & Non-Goals

3.1 Non-negotiable principles

1. Risk authority is absolute. Memory, ML, sentiment, correlation, optimization and ranking cannot bypass a hard risk gate.
2. Data truth precedes intelligence. Invalid, stale, duplicated, misordered or schema-incompatible data cannot enter the executive decision path.
3. Execution truth is separate from strategy truth. A broker rejection or poor fill is not automatically a strategy failure.
4. Evidence authority is bounded. Small, stale or unstable historical samples must not materially reshape decisions.
5. Temporal causality is preserved. No feature, memory record, model output or journal field may use information that was unavailable at its stated timestamp.
6. No silent adaptation. Production-affecting behavior is attributable to a versioned code revision, policy, feature schema, model or calibration artifact.
7. Memory is durable but revisable in relevance. Drift reduces authority; it does not erase history.
8. Hypothetical outcomes remain hypothetical forever unless a separate observed event later proves them.
9. NO_TRADE is a first-class, successful decision outcome when constraints or evidence do not justify action.
10. Every operationally significant action must be reconstructable from persistent records and versioned inputs.

3.2 Explicit non-goals

- No guarantee of profit, win rate, Sharpe ratio or drawdown target.
- No autonomous lot sizing from raw historical win rate.
- No self-modifying production model updated on every trade.
- No causal claim from a single post-hoc outcome.
- No Phase 1 dependency on production ML, vector retrieval or complex adaptive learning.
- No destruction of historical memory merely because the market regime changed.
- No use of the frontend as the authoritative source of trading, risk or execution logic.
- No placement of multiple independent node_modules directories for separate frontend modules in the monolithic repository.
- No silent conversion of a hypothetical or simulated journal record into an observed live trade record.

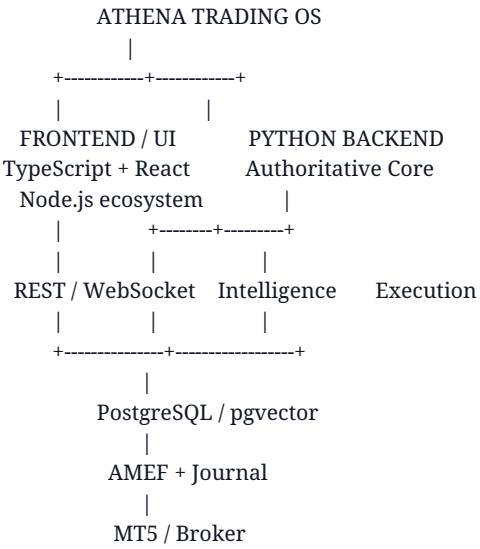
3.3 Authority hierarchy

HARD SAFETY / ACCOUNT GUARDIAN
> EXECUTION POLICY
> PORTFOLIO EXPOSURE
> DECISION POLICY
> STRATEGY / SMC / REGIME / ML
> AMEF EVIDENCE
> EXPLANATORY / RESEARCH SIGNALS

4. System Context & High-Level Architecture

ATHENA v2.7 is logically organized into five planes. The planes may initially run as modules in a small number of processes, but their contracts must remain separable so that deployment can evolve without changing decision semantics.

Plane	Responsibilities	Primary output
Data Plane	ticks, OHLCV, news/context, account telemetry, broker events, normalization, data quality	Validated timestamped events
Intelligence Plane	SMC features, deterministic regime, advanced regime, ML, calibration, AMEF retrieval, drift	Versioned EvidenceSnapshot and candidate context
Decision Plane	candidate activation, cost, exposure, ranking, decision attribution, risk policy evaluation	Executable or non-executable decision
Execution Plane	ExecutionIntent, broker adapter, order lifecycle, reconciliation, position management	Broker-truth state
Governance Plane	identity, permissions, policy versions, model registry, audits, deployment state, observability	Authority and traceability



4.1 Intelligence Plane versus Execution Plane

A major v2.7 hardening is that intelligence work is allowed to run asynchronously or from pre-computed snapshots. AMEF retrieval and ML inference are not allowed to become an unbounded latency dependency for a safety-critical order path. The execution plane consumes the latest valid EvidenceSnapshot and verifies its age, policy compatibility and authority before acting.

Implementation consequence
ATHENA can remain a monolithic application while the code behaves as if it were a set of well-defined services. This is preferable to premature microservices. Contracts come first; process separation can come later.

5. Frontend Trading OS Architecture

The frontend is a dedicated Trading OS application built with TypeScript and React. Node.js supplies the frontend toolchain and runtime ecosystem, and may expose a thin API/BFF boundary where required. The frontend is responsible for presentation, interaction, local UI state and authenticated user experience. Python remains authoritative for market state, risk, strategy and execution.

5.1 Frontend technology position

Component	Role

TypeScript	Strongly typed frontend contracts, API models, UI logic
React	Component and page architecture
Node.js ecosystem	Package management, build tooling, development server and optional BFF/gateway functionality
Single node_modules	One frontend dependency tree under frontend/
REST	Configuration, administrative queries and non-streaming operations
WebSocket / SSE	Live telemetry, journal updates, position and execution events
Charting layer	Interactive market charts and future backtesting canvas

5.2 Navigation model

```
ATHENA
|
+-- Dashboard
|
+-- Executioner
|
+-- Backtesting
|
+-- Journal
|
+-- Settings
|
+-- optional system/governance overlays
```

Persistent left-hand navigation. Main workspace is context aware.

5.3 Authentication experience

Authentication is a dedicated frontend module under frontend/src/authentication/. It is not mixed into dashboard components. The module contains the user-facing authentication journey and communicates with backend authentication endpoints.

```
frontend/src/authentication/
|-- pages/
|   |-- SplashScreen.tsx
|   |-- Login.tsx
|   |-- SignUp.tsx
|   `-- ForgotPassword.tsx
|-- components/
|-- hooks/
|-- services/
|-- types/
`-- routes/
```

5.4 UI design language

Element	Design rule
Background	Cybernetic Institutional Obsidian: deep obsidian, low-glare surfaces
Active/data highlight	Electric cyan
Bullish/profit	Emerald
Bearish/loss	Crimson
Trade opened in decision stream	Blue
Winning trade in decision stream	Yellow
Neutral/system	Cyan/white
Warning	Amber
Accessibility	Never rely on color alone; pair color with icon, label or state

	text
Typography	Compact, high-legibility data typography with restrained hierarchy

5.5 Frontend state model

The frontend should render server-authoritative state rather than re-derive trading state from raw UI events. Core displayed state should include system health, account state, instrument state, strategy state, current risk state, decision stream, journal lifecycle state and broker connection state.

6. Python Backend Architecture

Python is the authoritative implementation environment for ATHENA trading logic. The backend contains market ingestion, event orchestration, structural analysis, risk, strategy modules, execution adapters, persistence, AMEF and governance. The frontend never places an order directly.

6.1 Core backend responsibilities

- Event intake and normalization.
- Market and account state.
- Multi-timeframe SMC analysis.
- Deterministic and advanced regime classification.
- Scalping and day-trading strategy engines.
- Risk and account policy evaluation.
- Correlation and portfolio exposure evaluation.
- ML inference and calibration.
- AMEF retrieval and EvidenceSnapshot generation.
- Execution intent generation and broker adapters.
- Order lifecycle and reconciliation.
- Journal/event persistence and post-trade attribution.
- Health checks, metrics and deployment diagnostics.

6.2 Backend module boundary

```
backend/
|-- app/
|-- authentication/
|-- core/
|-- market/
|-- smc/
|-- regime/
|-- strategies/
|   |-- scalping/
|   `-- day_trading/
|-- ml/
|-- memory/
|-- risk/
|-- portfolio/
|-- execution/
|-- journal/
|-- data/
|-- telemetry/
|-- governance/
|-- api/
|-- tests/
`-- main.py
```

6.3 Frontend/backend boundary rules

Rule	Implementation
UI cannot be authoritative	Python returns authoritative server state
UI commands are intents	Frontend requests are validated by backend policy
No client-side lot authority	Volume is derived and approved in Python risk path
No UI-side broker credentials in trading logic	Credentials are handled through backend security boundary
Versioned contracts	API payloads reference schema/version identifiers where necessary
Real-time events are append-like	Frontend should tolerate reconnect and resume from server state

7. Account Management & Settings Architecture

Settings becomes a major control plane in v2.7. It is not merely a collection of cosmetic preferences. It expresses the policies that define how ATHENA is allowed to operate. The most important addition is policy inheritance: global defaults exist, but individual accounts can override them, and more specific policies can apply to strategies, assets or sessions.

7.1 Configuration hierarchy

```
GLOBAL DEFAULTS
    |
    +-- ACCOUNT POLICY
        |
        +-- STRATEGY POLICY
            |
            +-- ASSET POLICY
                |
                +-- SESSION / TEMPORARY POLICY

Effective Policy = highest-priority explicit override + inherited defaults
```

Important
Overrides must be explicit and auditable. The UI should show where every effective value came from: Global, Account, Strategy, Asset or Session. A user should never wonder why a number is different from the global default.

7.2 Account profile

- Account display name and account identifier.
- Broker and server.
- Account type: personal, demo, funded or other supported classification.
- Base currency and leverage metadata.
- Enabled/disabled state.
- Allowed instruments and strategy modules.
- Account-specific risk policy.
- Account Guardian state and current drawdown measurements.
- Credential references; secrets themselves remain protected and never displayed as plain text.

7.3 Risk settings

Setting	Purpose	Override scope
Risk per trade	Base risk allocation	Global / account / strategy / asset
Maximum daily drawdown	Hard daily loss ceiling	Global / account
ATHENA safety drawdown	Internal stricter protection threshold	Account
Maximum overall drawdown	Hard total drawdown ceiling	Account

Maximum concurrent positions	Portfolio safety	Global / account
Maximum correlated exposure	Portfolio risk	Account / asset group
Maximum consecutive losses	Circuit breaker	Global / account / strategy
Daily trading lock	Prevent further entry after lock condition	Account
Emergency stop	Immediate no-new-entry/halt action	Global / account / module
Maximum spread	Instrument execution safety	Asset
News blackout policy	High-impact event protection	Global / account / asset

7.4 Funded-account configuration example

A funded account can be configured with internal protection thresholds that are deliberately stricter than the external program requirements. ATHENA stores both the external reference rule and its own execution safety rule. The external rule is informational/policy context; the ATHENA threshold is the active safety authority.

Account: Funded-100K
External daily DD reference: configured value
ATHENA daily DD safety limit: stricter configured value
External overall DD reference: configured value
ATHENA overall DD safety limit: stricter configured value
Risk per trade: account-specific
Maximum positions: account-specific
Trading lock: enabled

7.5 Other Settings domains

Domain	Representative settings
Strategies	Enable scalping/day trading, strategy priority, timeframes, setup thresholds
Instruments	Enabled symbols, symbol mappings, sessions, spread limits, volatility policy
Sessions	Market session windows, preferred trading windows, session-specific restrictions
Correlation	Currency groups, pair correlation rules, cluster exposure
News	Tier rules, blackout windows, severity policy
Notifications	Desktop, email/webhook later, event subscriptions
Audio telemetry	Sound profile, per-event enable/disable, volume, quiet mode
Appearance	Theme, compact density, chart preferences, accent behavior
Language	Locale selection, translation packs, number/date format
Security	Session timeout, 2FA readiness, device/session visibility
System	Backend health, database status, AMEF status, version, diagnostics
Research	Backtest defaults, dataset versions, cost model profiles

8. Market Context & Regime Engine

Layer 1 answers a contextual question: what market environment is ATHENA in, which engines are permitted, and what conditions are unsafe? It does not decide the exact entry by itself.

8.1 Deterministic Phase 1 context

- Configured higher-timeframe structure state.
- Basic trend/range/transition/unknown classification from deterministic rules.
- ATR/volatility state.
- Session state.
- Spread and data-quality state.
- News blackout state where a validated calendar source exists.

- Account risk state.
- Current open exposure and strategy module state.

8.2 Regime states

Regime	Characteristics	Default behavior
TREND	Directional HTF structure and aligned momentum	Day trading and selected pullback/scalp observations
RANGE	Bounded price behavior with controlled volatility	Scalping/mean-reversion logic when enabled
TRANSITION	Conflicting structure or regime change in progress	Reduce activity; require confirmation
HIGH_IMPACT_VOLATILITY	Extreme event or volatility shock	HARD HALT / no new entries by policy
UNKNOWN	Insufficient or invalid context	NO_TRADE

8.3 Advanced regime intelligence

Later phases may introduce probabilistic regime models, hysteresis, transition age, regime stability scores and drift detection. The advanced system may improve classification but cannot bypass safety rules. Every regime result is versioned.

9. Scalping & Day Trading Engines

ATHENA is designed to run scalping and day-trading engines concurrently when both are enabled and the current environment permits them. Concurrent evaluation does not mean concurrent execution. The portfolio/risk layer still arbitrates all candidates.

9.1 Scalping

- Uses lower-timeframe structure for entry precision.
- Consumes higher-timeframe context as a directional and risk filter.
- Looks for micro pullbacks, liquidity events, OB/FVG reactions, CHoCH/BOS and displacement.
- Uses configured execution costs, spread and latency constraints.
- Has an isolated loss/recovery state machine.

9.2 Day trading

- Uses HTF directional context and pullback logic.
- May remain active in a trending market even while scalping also operates.
- Has its own risk state and should not inherit scalping recovery multipliers automatically.
- Uses broader holding-period assumptions and separate execution/management rules.

9.3 Candidate object

Field	Meaning
candidate_id	Stable candidate identifier
asset	Instrument
module	Scalping / Day trading
direction	BUY / SELL
structural_context	HTF/LTF SMC evidence
regime_context	Current regime and confidence
entry_zone	Candidate zone/POI
invalidation	Structural or price invalidation
target_model	Configured target logic
expected_net_value	Expected value after friction
risk_eligibility	Account/portfolio eligibility

decision_state	OBSERVE / READY / BLOCKED / EXECUTE

10. ATHENA Memory & Experience Fabric (AMEF)

AMEF is the persistent experience layer. It stores what ATHENA observed, what it inferred, what it decided, what alternatives existed, what the broker actually executed, what happened afterward, and the defensible attribution associated with the episode. AMEF is contextual evidence, not a replacement for the deterministic risk engine.

10.1 Memory domains

Domain	Examples	Primary use
Market State	Regime, session, volatility, spread, liquidity, news	Find comparable environments
Structural	BOS/CHoCH/OB/FVG lifecycle, retests, invalidations	Measure POI quality
Decision	Candidate set, votes, selected/rejected candidate, reasons	Learn decision quality
Execution	Fill status, latency, slippage, broker errors	Separate execution quality
Outcome	R, MAE, MFE, time-to-event, exit path	Evaluate realized response
Counterfactual	Bounded rejected candidate path	Opportunity-cost analysis
Failure Patterns	Recurring adverse contexts	Warnings and policy tightening
Success Patterns	Recurring favorable contexts	Supporting evidence
Journal State	Fields observed over time	Human-readable audit and debugging

10.2 Canonical EpisodeRecord

```
EpisodeRecord
  identity
  market_context
  structural_context
  regime_context
  model_context
  decision_context
  execution_context
  outcome_context
  counterfactual_context
  attribution_context
  journal_context
  memory_metadata
```

10.3 AMEF retrieval contract

11. Validate schema and lineage compatibility.
12. Apply asset/module and temporal compatibility filters.
13. Exclude invalidated or disqualified records from normal retrieval.
14. Compute explicit structural and regime similarity components.
15. Apply recency and temporal decay.
16. Apply execution relevance only where relevant.
17. Normalize weights across retrieved evidence.
18. Compute raw count and effective sample size.
19. Compute evidence authority independently from evidence bias.
20. Return an auditable EvidenceSnapshot with record IDs, scores, weights, authority and policy versions.

10.4 Similarity and evidence formulas

```
S_SIM = w_r*S_REGIME + w_s*S_STRUCTURE + w_t*S_TIMEFRAME
    + w_m*S_MOMENTUM + w_v*S_VOLATILITY + w_x*S_EXECUTION_CONTEXT

W_i = S_SIM^a * S_REGIME^b * R_RECENCY^c * Q_DATA^d * Q_EXEC^e

N_effective = (sum(W_i))^2 / sum(W_i^2)
```

10.5 Evidence authority states

State	Authority behavior
COLD_START	Authority = 0. Retrieval may be logged in shadow mode
LIMITED	Telemetry/ranking context only; no sizing authority
ESTABLISHED	May influence candidate scoring within configured caps
DEGRADED	Authority automatically reduced
INVALIDATED	Authority = 0; excluded from normal retrieval

10.6 Memory poisoning safeguards

- Quality states for every experience record.
- No silent overwrite of observed records.
- Retroactive evidence may suppress authority without rewriting the historical event itself.
- Vector index rebuilds must reproduce retrieval from canonical records.
- Counterfactual records carry an immutable hypothetical flag.
- Memory cannot bypass hard risk gates or directly define lot size.
- Cold-start memory authority is zero.

11. POI / Order Block Lifecycle

Order Blocks, FVGs and other POIs are stateful entities with a lifecycle. The journal and AMEF should record the lifecycle so ATHENA can later analyze whether a zone formed cleanly, reacted, retested, degraded or invalidated.

```
DETECTED
  -> CONFIRMED
  -> ACTIVE
  -> MONITORING
  -> REACTED
  -> RETESTED
  -> DEGRADED
  -> INVALIDATED
  -> ARCHIVED
```

11.1 POI fields

Field	Description
poi_id	Stable identifier
type	OB / FVG / liquidity zone / other
asset	Instrument
timeframe	Formation timeframe
formation_timestamp	Timestamp at formation
bounds	Price range
direction	Bullish / bearish
strength	Versioned structural score
freshness	Retest/decay metric
state	Current lifecycle state
reaction_metrics	MFE/MAE/time-to-reaction

invalidation_reason	Canonical reason code
memory_statistics	Historical evidence summary

11.2 Dynamic zone evidence

Historical evidence may modify the confidence applied to a POI, but it never silently overwrites the base structural score. The adjustment is bounded, signed and auditable.

```
S_SMC_ADJUSTED = clamp(S_SMC_BASE * (1 + alpha * EvidenceBias), 0, 1)
```

12. Decision, Attribution & Counterfactual Learning

12.1 Tripartite attribution

Dimension	Question
Decision Quality	Was the decision reasonable using the information available at decision time?
Execution Quality	Did the broker execution match the intended payload acceptably?
Market Outcome	What did price do after execution or hypothetical entry?

12.2 Why Trade? / Why Not?

Every candidate should have a machine-readable explanation object. When a trade is rejected, the rejection reason is as important as a fill. The Executioner should surface both positive evidence and blocking evidence.

```
WHY TRADE
  HTF bias aligned
  LTF confirmation present
  POI valid
  Net value acceptable
  Account risk allowed
  Portfolio exposure allowed

WHY NOT
  Daily DD too high
  Spread too wide
  News blackout
  Insufficient LTF confirmation
  Stale evidence snapshot
  Broker state unsafe
```

12.3 Counterfactuals

ATHENA may track a bounded hypothetical path for rejected/unselected candidates using an explicit cost model. Such records remain hypothetical permanently. They are used to measure false rejection and opportunity cost without pretending an unexecuted trade was held.

- Only Top-K materially eligible rejected candidates should normally be simulated.
- Each simulation uses the actual decision timestamp and explicitly versioned market data/cost assumptions.
- Counterfactual results cannot be written into observed execution fields.
- Counterfactual authority is discounted relative to observed execution records.
- Counterfactual ablation is required before such evidence influences production authority.

13. Machine Learning & Calibration

ATHENA v2.7 retains a probabilistic ML layer, with XGBoost as the initial candidate model family, but the implementation sequence is conservative. ML begins in shadow mode after data contracts and deterministic context are stable.

13.1 Model lifecycle

21. Generate versioned features from validated data.
22. Create versioned labels using time-safe definitions.
23. Train with time-ordered data and leakage controls.
24. Calibrate probabilities on out-of-sample data.
25. Evaluate by regime, session, volatility and asset where applicable.
26. Deploy challenger in shadow mode.
27. Compare champion versus challenger using predefined acceptance gates.
28. Promote only through governed release process.
29. Monitor drift and calibration degradation.
30. Revert to champion or deterministic fallback when policy demands.

13.2 Calibration requirements

- Time-ordered evaluation.
- Minimum sample floors.
- Explicit calibration method/version.
- Fallback probability policy when calibration is unavailable.
- Calibration drift monitoring.
- Model and calibration versions stored in every decision record.
- No direct position sizing from raw uncalibrated probability.

13.3 Shadow-first principle

Promotion rule
A model can influence production execution only after the deterministic baseline is reproducible and the model passes out-of-sample, calibration, replay and governance gates. Shadow mode is not a cosmetic UI feature; it is a formal safety stage.

14. Risk Governance & Account Guardian

The Risk layer is the final authority before execution. v2.7 introduces the Account Guardian concept to make account-level protection visible and enforceable independently of individual strategies.

14.1 Account Guardian responsibilities

- Continuously compute balance, equity and drawdown state.
- Apply account-specific daily and overall drawdown limits.
- Apply stricter ATHENA safety thresholds where configured.
- Enforce daily locks and session locks.
- Track strategy/module risk separately.
- Aggregate current open exposure.
- Prevent execution when account state is unknown or unsafe.
- Publish explicit reason codes to Dashboard, Executioner and Journal.

14.2 Risk state machine

```
NORMAL
  -> LOSS_1
  -> LOSS_2
  -> COOLDOWN
  -> RECOVERY
  -> NORMAL

Any hard safety violation
  -> LOCKED / HALT

Account daily DD threshold
  -> DAILY_LOCK

Overall DD threshold
  -> ACCOUNT_LOCK
```

14.3 Position sizing

Sizing remains deterministic and bounded. The effective risk budget is computed only after strategy eligibility, account policy, exposure and hard gates are satisfied. Memory evidence does not directly increase lot size.

```
RiskBudget = min(GlobalLimit, AccountLimit, StrategyLimit, AssetLimit, RemainingDailyRisk, ExposureCapacity)
PositionSize = f(RiskBudget, StopDistance, ContractSpecification, ExecutionPolicy)
```

15. Correlation, Currency Exposure & Opportunity Ranking

As ATHENA expands beyond XAUUSD, risk must be evaluated at portfolio level. A candidate can be individually attractive and still be rejected because it adds too much exposure to an already concentrated portfolio.

15.1 Exposure layers

- Instrument exposure.
- Currency exposure for FX.
- Asset-cluster exposure.
- Highly correlated position exposure.
- Pending candidate exposure.
- Account-specific risk capacity.

15.2 Opportunity score

```
S_opp = E[R_net] * P_calibrated * RegimeFitScore * RiskEligibility
```

RiskEligibility is a gate, not an invitation to rank unsafe candidates more highly. A candidate with RiskEligibility = 0 is not executable.

16. Execution & Broker Interfaces

ATHENA produces a broker-neutral ExecutionIntent. Broker adapters translate that intent into broker-specific requests. Broker-specific behavior must not leak backwards into strategy logic.

Contract	Required fields
ExecutionIntent	decision_id, asset, side, volume/stake, requested price, SL/TP policy where applicable, expiry where applicable, max slippage, policy versions
BrokerResult	broker_order_id, status, requested values, fill values, filled volume, slippage, latency, broker code, retry/reconnect state
ReconciliationResult	order state, broker state, internal state, mismatch reason, final reconciliation status

16.1 Execution truth rules

- A missing broker acknowledgement is not a fill.
- An order rejection is an execution-system condition, not automatically a strategy loss.
- Partial fills must be represented explicitly.
- Retries must be bounded and auditable.
- Reconnect must reconcile broker state before new entry authority is restored.
- Execution latency and slippage are captured for later attribution.

17. Data, Persistence & Schema Lineage

PostgreSQL is the canonical transactional source of truth. pgvector is the preferred initial retrieval implementation for AMEF because it keeps structured metadata, transactional records and similarity search within one durable platform. The vector layer remains rebuildable and may not be treated as the only copy of memory.

Store	Purpose	Rule
PostgreSQL	Users, accounts, configuration, orders, trades, decisions, episodes, POI states, journal events, audit	Canonical source of truth
pgvector	Similarity retrieval over normalized embeddings/features	Rebuildable acceleration layer
Raw/derived files	Market history, retained raw feeds, feature datasets	Versioned / append-only where practical
Model registry	Champion/challenger artifacts, hashes, metrics	Immutable artifact metadata
Logs	Operational, execution, risk and audit telemetry	Rotation + retention policy

17.1 Version lineage

Artifact	Required lineage
Episode	episode schema, feature schema/hash, regime policy, memory policy
Decision	decision schema, feature hash, configuration version, decision policy
Prediction	model version, feature schema, calibration version, prediction timestamp
Backtest	dataset version, feature schema, cost model, strategy version
Deployment	Git commit, artifact hash, environment, migration version

17.2 Event and idempotency rules

- Every externally meaningful event has a stable event_id.
- Duplicated delivery must not create duplicate financial state transitions.
- Order, fill and reconciliation state changes are appendable transitions where practical.
- Decision timestamps use UTC and retain both source timestamp and ingestion timestamp.
- Clock skew is monitored and abnormal timestamps are quarantined.

18. Event-Driven 15-Stage Decision Pipeline

Stage	Responsibility	Policy
1 Event Intake	Ticks, bars, news, account telemetry, UTC	Invalid/stale event can stop path

	normalization	
2 Safety Gates	Spread, news, drawdown, heartbeat, data quality	HARD STOP
3 Feature / SMC	HTF structure, BOS/CHoCH, OB/FVG, momentum	Feature computation
4 Regime	Stable regime, transition, confidence	UNKNOWN/HIGH_VOL can stop
5 Candidate Generation	Scalping/day engines produce candidates	Module-gated
6 Evidence Snapshot	AMEF retrieval and evidence preparation	Advisory unless authority is established
7 ML Evaluation	Raw and calibrated probability, MAE/MFE estimates	Policy-dependent
8 Cost & Friction	Spread, slippage, commission, latency estimate	Reject if net value fails
9 Exposure & Correlation	Portfolio and currency limits	HARD STOP / rejection
10 Opportunity Ranking	Compare eligible candidates	No hard-gate authority
11 Risk State / Sizing	Circuit breaker, bounded sizing	HARD STOP / bounded sizing
12 Decision Attribution	Selected/rejected candidates and reasons	Audit requirement
13 Execution Dispatch	Create intent, send, acknowledge, reconcile	Execution failures distinct
14 Outcome Attribution	R, MAE, MFE, execution quality, POI outcome	Non-causal language
15 Memory Write & Journal	Persist episode, lifecycle, telemetry	Audit persistence failure may halt

```
EVENT
 -> SAFETY
 -> CONTEXT/SMC
 -> REGIME
 -> CANDIDATES
 -> EVIDENCE SNAPSHOT
 -> ML
 -> COST
 -> EXPOSURE
 -> RANK
 -> RISK
 -> DECISION
 -> EXECUTION
 -> OUTCOME
 -> MEMORY/JOURNAL
```

19. Observability, Executioner & Decision Stream

The Executioner tab is an operational command center. The new design uses instrument tabs so the user can move quickly between active assets while preserving a consistent decision-stream view.

19.1 Instrument tab design

```
[ XAUUSD ] [ EURUSD ] [ GBPUSD ] [ USDJPY ] [ AUDUSD ] [ USDCAD ] ...

Selected instrument: XAUUSD
Status: SCANNING / WAITING / READY / POSITION OPEN / BLOCKED
Regime: TREND_BULLISH
Risk: ACCOUNT_SAFE
```

19.2 Analysis / Decision Stream

The stream should show structured events that are meaningful for debugging and operational review. It should not be described as a verbatim dump of hidden model reasoning. Each line should have a timestamp, event type, concise explanation, evidence references and state transition when applicable.

```
08:02  ANALYSIS
    H4 structure = bullish BOS
    HTF POI = bullish OB
    Confidence = 0.87
```

09:13 ANALYSIS
M15 CHoCH confirmed
Waiting for M5 confirmation

09:21 DECISION
Candidate validated
Risk policy = approved

09:22 POSITION OPENED
BUY XAUUSD
Risk = 0.25%

10:17 WIN
TP reached
Realized = +2.8R

19.3 Decision stream semantics

Event	Display intent
ANALYSIS	Green text for normal analytical observations
POSITION OPENED	Blue event marker
WIN	Yellow event marker
LOSS	Red event marker
WARNING	Amber event marker
BLOCKED	Distinct blocked-state marker plus reason
ERROR	Critical red marker with subsystem and error code
SYSTEM	Neutral/cyan system telemetry

19.4 Decision timeline

SCAN -> HTF BIAS -> POI -> LTF CONFIRMATION -> RISK CHECK -> ENTRY -> MANAGEMENT -> EXIT -> ATTRIBUTION

The timeline should allow a user to click into a stage and recover the exact journal/evidence snapshot associated with the event. This creates a direct bridge between live monitoring and later debugging.

19.5 Why Trade? and Why Not? panels

- Why Trade? lists the strongest evidence that supported the selected candidate.
- Why Not? lists blocking conditions and rejected alternatives.
- Both panels are generated from structured reason codes plus human-readable translations.
- Rejected candidates may be linked to counterfactual records later.

20. Journal & Continuous Trading Record

ATHENA should not simply copy the user spreadsheet into a database. The uploaded trading journal establishes the methodology fields: date/session, HTF trend and structure, BOS/OB, LTF trend and CHoCH, LTF OB/FVG, entry zone, entry/SL/TP, R:R, outcome, exit reason, mistakes and lessons. v2.7 re-engineers these ideas into a living, timestamped journal lifecycle that fills fields as observations become available.

20.1 Journal principle

The journal starts before the trade.
When ATHENA starts, it scans the saved asset universe. The journal records that the instrument was scanned, its current context and what is known. Fields such as LTF POI remain explicitly pending until that condition actually occurs.

20.2 Journal lifecycle

DISCOVERED
-> SCANNED
-> HTF ANALYSIS
-> HTF POI IDENTIFIED
-> LTF MONITORING
-> LTF STRUCTURE DETECTED
-> LTF POI FORMED
-> ENTRY QUALIFICATION
-> RISK VALIDATION
-> TRADE EXECUTED
-> POSITION MONITORING
-> TRADE CLOSED
-> POST-TRADE ANALYSIS
-> EXPERIENCE / AMEF

20.3 Living journal fields

Field family	Examples	Population behavior
Session / time	Date, session, timestamp	Filled at scan and updated by event
HTF context	Trend, structure, BOS, HTF OB	Filled when confirmed
LTF context	Trend, CHoCH, LTF OB, FVG	Remains pending until observed
Entry model	Entry zone, entry, SL, TP, R:R	Filled when candidate becomes executable
Risk	Risk %, account state, exposure	Filled at validation
Execution	Requested price, fill, slippage, latency	Filled from broker result
Outcome	WIN/LOSS, R, MAE, MFE, exit reason	Filled after position lifecycle ends
Quality	Decision quality, execution quality	Post-trade attribution
Mistakes	Rule violation / deviation / none	Generated conservatively after analysis
Lessons	Evidence-based lesson summary	Generated from attribution and AMEF
AMEF	Episode ID, EvidenceSnapshot, authority	Linked throughout lifecycle

20.4 Journal states for pending information

ATHENA must distinguish between values that are not applicable and values that are not yet observable. Recommended states are PENDING, OBSERVED, CONFIRMED, INVALIDATED, NOT_APPLICABLE, and BLOCKED.

20.5 Two journal views

View	Purpose
Market Journal	Continuous observations for scanned instruments and developing setups
Trade Journal	Executed trade records with entry, management, exit and outcome
Decision Audit	Every significant selected/rejected decision and reason
Research / AMEF	Experience episodes, memory evidence and learning attribution

20.6 Daily automated behavior

31. Load saved enabled instruments and account profiles.
32. Create daily scan session and per-instrument journal context.
33. Record current HTF and deterministic context for each instrument.
34. Update journal fields when structural events occur.
35. Create candidate/setup entries when a qualifying POI appears.
36. Record risk validation and the reason for approval or rejection.
37. Write execution details when a position is actually opened.

38. Continue logging management events until closure.
39. Complete post-trade attribution and link the record to AMEF.
40. Generate end-of-day summary and preserve all events for replay.

21. Backtesting & Research Platform

Backtesting is the verification environment for strategy logic, memory logic, cost assumptions and learning governance. It must replay the same data contracts used by live execution so that backtest decisions are structurally comparable to live decisions.

21.1 Validation modes

- time-ordered train/validation/test splits
- walk-forward evaluation
- regime-segmented evaluation
- transaction-cost and slippage sensitivity
- Monte Carlo / trade-sequence robustness
- memory-on vs memory-off ablation
- calibration evaluation
- execution failure simulation
- counterfactual validation with hypothetical/observed separation
- latency and stale-snapshot simulation
- schema migration and replay tests

21.2 Validation ladder

Mode	Adaptive components	Purpose
A	ML OFF, AMEF OFF	Prove deterministic core
B	ML SHADOW, AMEF SHADOW	Measure without influencing decisions
C	AMEF advisory, ML shadow/advisory	Test evidence lift without executive risk
D	Limited executive authority	Controlled capital impact after formal gates
E	Fully governed production	Ongoing monitoring and promotion rules

21.3 Interactive backtesting UI

- Chart with entry/exit markers and SMC overlays.
- Historical replay synchronized to journal and decision stream.
- Stage-by-stage decision inspection.
- Memory-on vs memory-off comparison.
- Regime and session breakdowns.
- Execution friction dashboards.
- Counterfactual opportunity analysis.
- Exportable experiment metadata and lineage.

22. CI/CD, Git, Deployment & Operations

Delivery discipline is part of the architecture, not an afterthought. Production state must survive deployments, restarts and rollback. The system should be developed as a modular monolith and deployed as a coherent release artifact until operational evidence justifies further decomposition.

22.1 Git branches

Branch/artifact	Meaning

main	Stable, production-tested branch
dev	Integration branch
feature/*	Isolated work
release/*	Formal acceptance candidate
model artifact	Immutable model file + hash + version + evaluation metadata

22.2 GitHub Actions sequence

41. Checkout and dependency/lockfile integrity check.
42. Linting and formatting validation.
43. Static import and syntax checks.
44. Unit tests.
45. Contract/schema tests.
46. Database migration tests.
47. Broker adapter integration tests using mocks.
48. AMEF persistence and index-rebuild tests.
49. Replay/simulation tests.
50. Build artifact and environment verification.
51. Deploy only after required merge/approval gates.
52. Run post-deploy health and heartbeat checks.
53. Record deployment state and rollback metadata.

22.3 State-safe deployment

- Persistent volumes contain transactional data, AMEF records, vector indexes, raw/derived data, model registry and audit logs where retained.
- Application restart never deletes or recreates persistent state as a side effect.
- Migrations are explicit, versioned and backward-aware.
- Rollback is a code/artifact operation, not an instruction to restore old application data blindly.
- Vector indexes are rebuildable from PostgreSQL canonical data.

23. Testing & Validation

Test layer	Required scope
Unit	SMC scores, volatility metrics, thresholds, state transitions, reason codes
Contract	ExecutionIntent, BrokerResult, EpisodeRecord, EvidenceSnapshot, ModelMetadata
Integration	PostgreSQL, pgvector, broker mocks, event bus, migrations
Simulation	Deterministic market event streams and replay
Backtest	Walk-forward, costs, calibration, memory-on/off
Chaos / resilience	Broker disconnect, DB restart, stale feed, AMEF outage, model timeout
Security	Secrets, sessions, authorization boundaries, audit integrity
Deployment	Restart safety, rollback, migration integrity

23.1 Mandatory safety tests

- News blackout blocks execution whenever policy requires it.
- Daily drawdown threshold blocks new execution whenever reached.
- Overall drawdown threshold locks the account whenever reached.
- Invalid timestamps cannot enter decision features.
- Missing broker acknowledgement cannot become a fill.

- AMEF retrieval failure cannot corrupt canonical memory.
- Schema/model incompatibility fails closed for executive use where required.
- Memory retrieval cannot bypass hard gates.
- Deployment restart preserves positions, journal records, configuration and memory.
- Counterfactual records cannot populate observed execution fields.

24. Failure Modes & Degraded Operation

Failure	Detection	Default response
Stale feed	Timestamp age / heartbeat	NO_TRADE or HALT
Spread spike	Spread ratio to baseline	Reject / suspend entries
Broker disconnect	Adapter heartbeat	No new entries; reconcile after reconnect
Order rejection	Broker status	Execution failure attribution
AMEF unavailable	Service health / timeout	Memory-off safe mode if policy permits; otherwise NO_TRADE
Vector index unavailable	Index health	Use canonical store/cached valid snapshot if safe
Model unavailable	Inference timeout / health	Shadow/off or deterministic fallback per policy
Database unavailable	Connection health	Halt audit-critical execution if state cannot be confirmed
Clock skew	Source vs ingestion timestamps	Quarantine affected events
Schema mismatch	Compatibility check	Fail closed for executive path
Frontend disconnected	WebSocket heartbeat	Continue backend according to policy; UI reconnects and resynchronizes

24.1 Degraded-mode principle

A subsystem may fail without collapsing the whole system only when the resulting behavior remains safe and auditable. Every degraded mode must have an explicit policy. Unknown is not equivalent to healthy.

25. Security & Access Control

25.1 Authentication

- Splash -> Login / Sign Up / Forgot Password flow.
- Backend-issued sessions/tokens are authoritative.
- Protected frontend routes prevent unauthenticated access to trading controls.
- Authorization checks are enforced again on the backend; frontend route protection is not sufficient.
- Credential secrets are stored securely and are never logged in plaintext.

25.2 Access scopes

Role/capability	Representative permissions
Viewer	Read dashboard, execution state, journal and research
Operator	Start/stop modules, inspect positions, acknowledge operational controls
Administrator	Manage accounts, policies, credentials, system settings
Developer/Research	Access replay, diagnostics and research interfaces without production authority by default

25.3 Credential handling

Broker credentials, API secrets and authentication secrets must remain inside protected backend configuration and secret-management boundaries. The frontend may submit or request controlled credential operations, but it must not become the long-term secret store.

26. Performance & Governance Metrics

26.1 Operational metrics

- Broker round-trip latency.
- Order acknowledgement latency.
- Fill latency and slippage.
- Data freshness.
- Event processing delay.
- WebSocket event lag.
- Journal write latency.
- AMEF retrieval latency and cache hit rate.
- Account Guardian evaluation latency.

26.2 Trading and research metrics

- Expectancy and realized R.
- Profit factor, Sharpe and Sortino where statistically meaningful.
- Maximum drawdown and drawdown duration.
- MAE/MFE distributions.
- Win/loss and payoff distributions.
- Performance by regime/session/asset/module.
- Execution-adjusted versus decision-level outcome.
- Calibration error and probability reliability.
- Memory-on versus memory-off lift.
- False rejection / false acceptance and opportunity cost.

26.3 Governance metrics

- Percentage of decisions with complete lineage.
- Number of schema/model/policy incompatibility incidents.
- Number of hard-gate bypass attempts (target: zero successful).
- Percentage of counterfactual records correctly isolated.
- AMEF authority distribution by state.
- Number of production model promotions and rollback events.
- Degraded-mode frequency and duration.

27. Phased Development Roadmap

The roadmap is intentionally conservative. The key change from earlier roadmaps is that the deterministic context, authentication shell, account policy framework and live journal foundation appear earlier, while advanced adaptive learning remains later. This prevents ML/AMEF from being built on an unstable data and operational foundation.

Phase 1 - Foundation, Auth, UI Shell & Deterministic Context

- Git repository, branch policy, CI skeleton and local development environment.
- Modular monorepo with backend/, frontend/ and .venv/ at repository root.

- Frontend React/TypeScript shell with persistent sidebar and Cybernetic Institutional Obsidian design tokens.
- Authentication pages: SplashScreen, Login, Sign Up, Forgot Password.
- Backend authentication/session/authorization baseline.
- Dashboard and Settings shell.
- Global settings and account management model.
- Account Guardian with configurable daily/overall safety thresholds.
- XAUUSD configuration and MT5 handshake.
- Deterministic market context and basic regime state.
- Fixed baseline risk and safe dummy trade flow.
- Foundational journal session creation and event persistence.
- Oracle VPS health/heartbeat and state-safe deployment skeleton.
- No production ML, no executive AMEF authority.

Phase gate

Gate: authenticate, configure an account, connect to MT5, create a journal session, run a safe dummy trade, reconcile it, restart without state loss, and redeploy successfully.

Phase 2 - SMC Micro-Execution, Executioner & AMEF Shadow

- M1/M5 SMC feature detection and POI lifecycle.
- Scalping candidate generation.
- Day-trading engine skeleton with isolated risk state.
- Executioner instrument tabs.
- Structured Analysis/Decision Stream.
- Why Trade?/Why Not? reason model.
- EpisodeRecord capture.
- PostgreSQL canonical AMEF store + pgvector retrieval scaffold.
- EvidenceSnapshot generation in shadow mode.
- Base XGBoost model in shadow mode.
- Execution and decision attribution.
- Journal fields populated continuously as events occur.

Phase gate

Gate: data contracts stable, replayable episodes, AMEF rebuild works, Executioner is consistent with backend state, latency bounds pass, and ML/AMEF remain non-executive.

Phase 3 - Advanced Regime, Risk States & Controlled Evidence Authority

- Probabilistic regime engine, hysteresis and transition states.
- News blackout integration and enhanced macro context.
- Full risk state machine and recovery controls.
- AMEF authority states and capped influence.
- Calibration selection and monitoring.
- Portfolio exposure groundwork and correlation gates.
- Explicit degraded modes.
- Governance UI for model/policy/version status.

Phase gate

Gate: regime stability and drift tests pass; no hard-gate bypass; degraded modes are proven safe.

Phase 4 - Day Trading & Advanced Memory Learning

- Full HTF pullback day-trading module.
- POI quality analytics and historical lifecycle analysis.
- Counterfactual/rejected-opportunity records.
- Failure and success pattern aggregation.
- Decision/execution/market attribution reports.
- Drift monitoring such as ADWIN/Page-Hinkley where justified.
- Champion/Challenger offline loop.
- Memory-on versus memory-off experiments.

Phase gate

Gate: measurable out-of-sample behavior, attribution consistency, counterfactual isolation and no material degradation of deterministic baseline.

Phase 5 - Multi-Asset & Multi-Account Expansion

- 12 major FX pairs and selected indices.
- Per-asset profiles and per-account policy overrides.
- Currency exposure aggregation.
- Pairwise/cluster correlation gates.
- Asset-specific broker mappings and execution policies.
- Cross-universe opportunity ranking.
- Account dashboard with multiple account tabs/selection.
- Funded-account protection profiles.

Phase gate

Gate: per-asset validation, portfolio stress tests, broker mapping tests and account-policy isolation pass.

Phase 6 - Backtesting, Journal & Research Platform

- Interactive backtesting chart and replay.
- Full living journal viewer and filters.
- Decision timeline replay.
- Memory attribution views.
- Walk-forward and regime reports.
- Execution friction dashboards.
- Experiment lineage and export.
- Final operations documentation.

Phase gate

Gate: reproducible research environment with parity between live and replay contracts.

Phase 7 - Advanced Adaptive Intelligence

- Experience replay/EWC where justified by evidence.
- Sophisticated challenger models.
- Broader continual-learning research.
- Adaptive thresholds under formal governance.
- Advanced memory graph/semantic retrieval research if justified.
- Potential service extraction only where operational evidence warrants it.

Phase gate

Gate: explicit governance approval after stable production evidence; no automatic promotion based on complexity

alone.

27.1 Phase activation rule

A phase is complete only when functionality, telemetry, tests, replayability, documentation, failure handling and deployment behavior are verified. “Code exists” is not a definition of done.

27.2 Development order within each phase

```
DATA CONTRACTS -> DETERMINISTIC BEHAVIOR -> TESTS -> OBSERVABILITY -> REPLAY
-> SHADOW -> CONTROLLED AUTHORITY -> CAPITAL IMPACT -> EXPANSION
```

28. Definition of Done & Release Gates

The v2.7 implementation baseline is ready for development when every team can answer, in concrete terms: what information is available, what state ATHENA believes it is in, what evidence it is using, how much authority that evidence has, what risk policy applies, what it intended to do, what the broker actually did, and how the result will be written back without corrupting historical truth.

28.1 Release checklist

- All production-executive paths have version lineage.
- All hard safety rules have automated tests.
- Account Guardian state is observable and auditable.
- Frontend/backend contracts are versioned and synchronized.
- Authentication and authorization are enforced server-side.
- Journal state remains consistent through reconnect and restart.
- Executioner stream is derived from structured events, not arbitrary UI logs.
- AMEF can be rebuilt from canonical records.
- Counterfactuals are permanently separated from observed trade state.
- Backtests are reproducible with dataset and strategy lineage.
- Production model promotions are governed by champion/challenger evidence.
- Rollback and state preservation are tested.

Appendix A. Core Data Contracts

A.1 ExecutionIntent

```
{
  decision_id,
  asset,
  side,
  quantity_or_volume,
  requested_price,
  sl_policy,
  tp_policy,
  expiry_policy,
  max_slippage,
  account_policy_version,
  risk_policy_version,
  schema_version
}
```

A.2 BrokerResult

```
{
 broker_order_id,
 status,
 requested_price,
 fill_price,
 requested_volume,
 filled_volume,
 slippage,
 spread,
 latency_ms,
 broker_error_code,
 retry_count,
 reconciliation_state
}
```

A.3 EvidenceSnapshot

```
{
 snapshot_id,
 query_timestamp_utc,
 feature_version,
 regime_context,
 matches[],
 n_raw,
 n_effective,
 evidence_authority,
 evidence_bias,
 expiry_timestamp_utc,
 memory_policy_version
}
```

A.4 JournalEvent

```
{
 journal_event_id,
 journal_id,
 asset,
 timestamp_utc,
 state_before,
 state_after,
 field_updates[],
 evidence_refs[],
 reason_codes[],
 observed_or_hypothetical,
 feature_version,
 policy_version
}
```

Appendix B. Enumerations & Reason Codes

Category	Examples
Decision	BUY, SELL, MODIFY, NO_TRADE, OBSERVE, BLOCKED, HALT
Regime	TREND_BULL, TREND_BEAR, RANGE, TRANSITION, HIGH_VOLATILITY, UNKNOWN
Risk	NORMAL, LOSS_1, LOSS_2, COOLDOWN, RECOVERY, DAILY_LOCK, ACCOUNT_LOCK, SESSION_LOCK
POI	DETECTED, CONFIRMED, ACTIVE, REACTED, RETESTED, DEGRADED, INVALIDATED
Execution	FILLED, PARTIAL_FILL, REJECTED, CANCELED,

	BROKER_DISCONNECT, RECONCILIATION_FAIL
Outcome	TP_HIT, SL_HIT, TIME_EXIT, STRUCTURAL_EXIT, MANUAL_EXIT, MODULE_STOP
Failure attribution	REGIME_ERROR, LIQUIDITY_TRAP, MOMENTUM_CONTINUATION, COST_EROSION, EXECUTION_FAILURE, DATA_QUALITY
Memory	NO_SIMILAR_EVIDENCE, STRONG_SIMILARITY, FAILURE_PATTERN_MATCH, SUCCESS_PATTERN_MATCH, STALE_CONTEXT, INVALIDATED_EVIDENCE
Journal state	DISCOVERED, SCANNED, PENDING, OBSERVED, CONFIRMED, INVALIDATED, EXECUTED, CLOSED, ARCHIVED

Appendix C. Conceptual Repository Structure

```
athena/
|
|-- backend/
|   |-- authentication/
|   |-- api/
|   |-- core/
|   |-- market/
|   |-- smc/
|   |-- regime/
|   |-- strategies/
|   |   |-- scalping/
|   |   `-- day_trading/
|   |-- ml/
|   |-- memory/
|   |-- risk/
|   |-- portfolio/
|   |-- execution/
|   |-- journal/
|   |-- telemetry/
|   |-- governance/
|   |-- data/
|   |-- tests/
|   `-- main.py
|
|-- frontend/
|   |-- public/
|   |-- src/
|   |   |-- authentication/
|   |   |-- dashboard/
|   |   |-- executioner/
|   |   |-- backtesting/
|   |   |-- journal/
|   |   |-- settings/
|   |   |-- components/
|   |   |-- layouts/
|   |   |-- services/
|   |   |-- hooks/
|   |   |-- store/
|   |   |-- routes/
|   |   |-- types/
|   |   `-- utils/
|   |-- node_modules/
|   |-- package.json
|   |-- package-lock.json
|   |-- tsconfig.json
```

```
|   `-- vite.config.ts
|
|-- .venv/
|
|-- config/
|-- data/
|-- logs/
|-- scripts/
|-- tests/
|-- deploy/
|-- docs/
|-- .env
|-- .gitignore
`-- README.md
```

Environment rule
The Python virtual environment is a runtime dependency container, not the application repository. Backend and frontend are sibling application directories at repository root. The frontend has one node_modules directory. Python packages live in .venv.

Appendix D. Key Equations & Design Constraints

Continuous SMC score (conceptual):
S_SMC = w_TF*S_TF + w_Fresh*S_Fresh + w_Prox*S_Prox

Net expectancy:
E[R_net] = E[R_raw] - C_R

Opportunity score:
S_opp = E[R_net] * P_calibrated * RegimeFitScore * RiskEligibility

Bounded evidence adjustment:
S_adjusted = clamp(S_base * (1 + alpha*EvidenceBias), 0, 1)

Evidence similarity:
S_SIM = w_r*S_REGIME + w_s*S_STRUCTURE + w_t*S_TIMEFRAME
+ w_m*S_MOMENTUM + w_v*S_VOLATILITY + w_x*S_EXECUTION_CONTEXT

Evidence weight:
W_i = S_SIM^a * S_REGIME^b * R_RECENCY^c * Q_DATA^d * Q_EXEC^e

Effective sample size:
N_effective = (sum(W_i))^2 / sum(W_i^2)

D.1 Hard design constraints

- No look-ahead bias.
- No hard-gate override by memory or ML.
- No silent schema changes to historical episodes.
- No direct lot-size escalation from raw memory win rate.
- No deletion of historical episodes because of drift.
- No treating unfilled trades as executed trades.
- No causal certainty from post-hoc attribution.
- No production model replacement without promotion evidence.
- No deployment that can destroy persistent market memory.
- No executive use of expired EvidenceSnapshots.
- No transformation of hypothetical evidence into observed truth.

- No frontend authority over backend risk or execution.

Appendix E. Architectural Decisions Register

ID	Decision	Rationale
ADR-001	PostgreSQL + pgvector is canonical AMEF backend	Transactional truth, metadata filtering, auditability and vector retrieval in one platform
ADR-002	Intelligence Plane separated from Execution Plane	Prevents AMEF/ML latency from becoming an unbounded execution dependency
ADR-003	EvidenceSnapshot is execution-facing memory artifact	Makes freshness, lineage and reproducibility explicit
ADR-004	Deterministic baseline regime starts in Phase 1	Prevents later datasets from being regime-blind
ADR-005	Memory authority starts at zero during cold start	Prevents small-sample evidence from influencing execution
ADR-006	Observed and hypothetical evidence are separate	Prevents counterfactual leakage
ADR-007	Champion/Challenger remains production model gate	Prevents uncontrolled continual retraining
ADR-008	Schema + feature + model + policy hashes required	Preserves historical semantic integrity
ADR-009	TypeScript/React frontend; Python authoritative backend	Clear separation of UX from trading authority
ADR-010	Repository is a modular monolith	Simplifies development while preserving future service boundaries
ADR-011	One frontend node_modules at frontend/	Avoids dependency duplication and fragmented frontend environments
ADR-012	Journal is a living lifecycle record	Makes debugging and market-state reconstruction possible before and during trades
ADR-013	Executioner exposes structured decision traces	Provides auditability without treating raw hidden reasoning as a product surface
ADR-014	Settings uses hierarchical policy inheritance	Supports global defaults and stricter account-specific overrides

Appendix F. Review-Driven Change Log from v2.6

Review finding	v2.7 resolution
AMEF retrieval underspecified	Explicit compatibility filters, component similarity, normalized weights, N_effective and authority states
Memory poisoning risk	Quality states, invalidation, retroactive authority suppression, rebuildable index
Counterfactual ambiguity	Bounded Top-K, explicit cost policy, immutable hypothetical flag, discounted authority
Regime under-specification	Deterministic baseline, confidence, transition state, hysteresis roadmap and UNKNOWN rules
Drift mechanism vague	Explicit decay, authority reduction, challenger triggers and restoration policy
Calibration vague	Method evaluation, time ordering, sample floors, fallback and monitoring
SMC weights unjustified	Configuration versioning, sensitivity testing and no silent optimization
Execution attribution vague	Requested-vs-filled metrics and distinct attribution states
AMEF latency risk	Async/cached retrieval and execution-facing EvidenceSnapshot
Cold-start instability	Authority = 0 until evidence sufficiency/reliability gates pass

Schema/version risk	Lineage fields and compatibility checks
Missing memory-off baseline	Formal validation ladder A-E
Degraded mode missing	Subsystem failure matrix and fail-open/fail-closed policies
Frontend ambiguity	Rewritten as TypeScript/React/Node.js monolithic Trading OS with Python authority
Journal underused	Re-engineered as a live stateful observation/decision/trade record
Settings too shallow	Account-aware hierarchy, Account Guardian, risk, strategy, asset, session, security and UX configuration
Execution UI too opaque	Instrument tabs, structured Decision Stream, Why Trade?/Why Not?, timeline and semantic event colors

Final Architecture Statement

ATHENA v2.7 is the implementation baseline for the project. Future versions should extend v2.7 through controlled revisions rather than bypassing its contracts. The architecture is intentionally conservative about authority: hard risk controls are deterministic; intelligence is probabilistic; memory is contextual; learning is governed; execution is auditable; and historical experience is retained without being granted blind power.

The frontend is a Trading OS, not the trading engine. TypeScript/React delivers the operational experience, while Python remains the authoritative execution, risk and intelligence layer. The repository is a modular monolith: backend and frontend are sibling application directories, the Python virtual environment is separate from both, and frontend dependencies live in one node_modules directory.

The journal is not an after-the-fact spreadsheet. It is a living market record. ATHENA begins recording when an instrument is scanned, fills fields only when the relevant evidence exists, records rejected decisions as first-class outcomes, captures broker truth separately from strategy truth, and carries completed episodes into AMEF for future evidence retrieval and research.

The intended development sequence is therefore: establish data truth; establish deterministic safety; establish account policy and replayability; establish observation and journal capture; establish structured Executioner observability; establish calibration and shadow intelligence; establish controlled evidence authority; then expand to multi-asset and advanced adaptive intelligence. That sequencing is a core engineering feature of ATHENA.

Implementation motto
Build the foundation first. Measure everything. Remember context. Adapt carefully. Let risk remain the final authority.

Source baseline and design inputs: ATHENA v2.6 Master Architecture & Engineering Specification; existing ATHENA v2.7 baseline in the project file library; uploaded personal Trading Journal.xlsx as the methodological journal reference. The journal template is treated as a source of fields and workflow ideas, not as a schema to copy literally.